Spring 2026 H. Hakimzadeh	C435 Operating Systems ULTIMA 2.0 PHASE I – Scheduler and Semaphore	30 points Consult Canvas for Due Date

ULTIMA is a simulated cooperative (non-preemptive) multitasking operating system. Currently this system lacks many necessary operating system components. Your job for the remainder of this semester is to complete each component and develop a fully functional multi-tasking environment.

PHASE 1: Scheduler and Semaphore

The current system lacks any kind of process management or resource synchronization. This problem is clear when two tasks try to use a shared resource such as the screen (although we experimented with pthread mutex in lab 7 to see how this could be done). In general, our operating system must provide a mechanism for controlling access to any shared resource, such as printer, memory or disk (although in lab 10 and 12 we experimented with building semaphores to allow for mutual exclusion and synchronization). In addition, we really don't know which task is "actually" running, which task has been blocked, etc. (although in lab 12 we create a simulation to envision how this might be done)

Our job during this phase is to implement the notion of a binary SEMAPHORE. Of course, we must be able to support any number of semaphores. This is critical, since future phases of this project introduce new resources which will certainly need mutual exclusion and synchronization.

In addition to implementing the semaphore, we also need a scheduler with its own process table (more accurately a thread table). The thread table **should be implemented dynamically, for example a linked list of Task Control Block's (TCB**). At this point, the TCB's are needed to preserve the STATE of each Task. For the purpose of this project, we will maintain 4 states: (RUNNING, READY, BLOCKED, DEAD). In the future phases, TCB's will maintain other information such as a pointer to memory allocated by each task, file descriptors, and task priority.

One way to implement the Thread Table is to create a TCB for each task when it is being created and insert the task ID, name, and other relevant information in the TCB. Then insert the TCB in a scheduling ring or linked list.

```
int Create_Task(char *task_name, etc...);     // return the T_id
```

In short, we need at least two classes. One that implements a scheduling ring and the associated algorithms and the other for implementing semaphores. Develop clear and concise test cases to show how and why your system works. Plan these cases as a group. Each class that you implement must contain one or more debugging functions (e.g., dump()) which print the current state of that class. The overall phase will leverage these debugging functions to show that the phase is working.

```
Class scheduler {
        TCB *process_table;
        create_task();          // create appropriate data structures and calls coroutine()
        kill_task();            // to kill a task (Set its status to DEAD)
        yield();                // strict round robin process switch.
        garbage_collect();      // remove dead task, free their resources, etc.

        dump(int level);        // debugging function with level indicating the verbosity of the dump include some
                                functions which will allow you to dump the contents of the process table in a readable
                                format.  See the expected output section (below) for suggestions.
}
```

```
Class semaphore {
        char resource_name [64]; // the name of the resource being managed
        int sema_value;          // 0 or 1 in the case of a binary semaphore
        queue *sema_queue;
        down();                  // get the resource or get queued!
        up()                     // release the resource

        dump(int level);         // include some functions which will allow you to dump the contents of the semaphore in
                                    a readable format.  See the expected output section (below) for suggestions.
}
```

Suggestions and Hints:

- Do not use a busy wait in Down (), instead block the task. (check the pThread library, and our prior labs)
- Create a STATE parameter in the Task Control block (TCB) and have the scheduler check to see if a process is READY, BLOCKED, or DEAD before scheduling.
- Be aware of deadlocks.

Expected Output:

The output from this phase must clearly show that the scheduler and process table work properly. In addition, you should design a test scenario which clearly shows that the semaphore is performing its job correctly. I suggest having two or three tasks try to acquire a resource (say write to a window) and show that one of them will get the resource and the others promptly get queued and their state changes to BLOCK or READY.

Sample Process Table Dump:

To show that your scheduler and process table operate correctly, develop a public dump() method which simply produces a table similar to the one below: (See lab 12 for possible implementation)

Task Name	Task ID	State	etc.
Task1	1	Ready	
Task2	2	Running	
Task3	3	Ready	

Sample Semaphore Dump:

To show that your semaphore is working properly, develop a public dump() method which shows the sema-value as well as the content of sema-queue. (See lab 12 for possible implementation)

```
Semaphore Dump:
Resource:     "Resource Name"        // i.e. printer, memory, etc.
Sema_value:   0                      // 0 or 1
Sema_queue:   T-id-->T-id -->....
```

Note:

- Develop an estimate for how much time this phase requires. Think about all the various components, classes, proof of concepts, external research, group meetings, etc, and associate an amount of time to each. Perhaps a minimum and maximum amount. (that should give a fair average estimate). Then depending on your own assessment of your team's experience, multiply your estimate by a factor (a factor of 2 or 3 is not uncommon). Now, you have a little better estimate of the time it will take to complete this phase. As you move forward, your ability to estimate will be refined.

Expected Output:

To show that your Scheduler and Semaphore is working properly, you must show a number of tasks executing in various states (ready, running, blocked, dead). Along with screen shots showing what is happening to those tasks in their output window. (don't forget to review labs 4 & 7 which integrate threads and curses windows)

Spend some time to think about the best approach for testing this phase. Be prepared to create short narrated video of your program execution. Also be prepared to demo your phase in zoom if asked by me.

Managing Your Workflow:

- Each group should create one main folder called ULTIMA, with the following sub folders. Please place this folder in a cloud space that is shared by your team members (and possibly me if I ask for it) so that everyone in your team has access to all the resources.

  ULTIMA
  - Introduction
    - Cover page with your group name and number, course # and name, project name, date, group names, e-mails, etc.
    - Project Abstract (Summary of accumulated phase abstracts. With each phase this will be supplemented)
    - Design diagram (Summary of accumulated phase design diagrams. With each phase this will be supplemented)
    - Resume from each team member (use the one from your profile submitted in discussion)

  - Phase 1 (Scheduler)
    - Copy of the assignment sheet
    - Phase Abstract
    - Design diagram for this phase (including the data structures and class hierarchy, nicely drawn using a drawing package)
    - External Documentation of your phase
    - Source files (with appropriate documentation, each class/module should have a primary author, as well co-authors, if appropriate)
    - A Word document showing the submission information including screenshots, code snippets, test cases, etc.
    - Each phase should be completely self-contained (all the files needed to compile and run the phase)
    - Video with narration for the phase.
  - Phase 2 (Memory Management)
  - Phase 3 (…)
  - Phase 4 (…)

What to hand in Canvas:

- A zipped copy of the appropriate folders for the phase:
  - Introduction folder
  - This Phase's folder

- A digital version of the completed Self and Peer evaluation form. (DO NOT place this form in the public area, it is not meant to be shared with your team members. This is your communication to me)

Guideline for each phase of the project

1) Cover Page

ULTIMA 2.0
CS-435 Operating System
Phase # - Phase Name

By:

Team Name and #

Team Members Up to 3 Members
(Names and e-mails)

Date

2) Phase Abstract (Short half a page abstract)
5) Phase Description (External Documentation)
6) Design Diagrams (Design diagrams. including the data structures and class hierarchy, nicely drawn using a drawing package)

7) Source Code

- Internal Documentation (Each function, Class, Struct, etc. should be documented)
- Nicely formatted (Use a smaller font if you copy your source code in a word document to minimize word wrap around)
- Copy of the make file and/or dependency graph of modules and files
- ".h", " ".cpp" , makefile (Separated by Modules)
    - Ultima.h, Ultima.cpp
    - Sema.h, Sema.cpp
    - Sched.h, Sched.cpp
    - Mem_mgr.h, Mem_mgr.cpp

8) Output (Must be coherent and annotated)

- Produce a test plan and include it in your output.
- Explain how and why your system is working.
- Explain what objectives your system should meet in order to be evaluated as a correctly functioning phase.
- Readable Dump() functions from class/data structure implemented in a phase (Scheduler, semaphore, memory manager, message handler, file system, and print spooler.)
- Screen dumps (nicely formatted and annotated by hand or electronically)

Self / Peer Evaluation Form
C435 - Operating Systems
ULTIMA 2.0

Your Name		Team Number:		Team Name:	
Date:		Phase Number:		Phase Title:	

What were your **exact responsibilities** for this phase?
A) Design Activity: (Name the functions, data structures, classes, algorithms you designed or help design)
B) Implementation Activity: (What functions, data structures, classes, algorithms you implemented or help implement)
C) Documentation Activity: (What functions, data structures, classes, or overall project did you document? Documentation could be classified as **internal** to a function, or to a class, or **external** about the phase or the overall project.)
D) Testing Activity: (What functions, data structures, classes, or overall project did you test?) Testing could be classified as unit testing (e.g., function or class) or overall/integration testing (phase level, or project level)
E) Other Activities: (not mentioned above, that you would like to highlight)

How would you improve the system you have implemented? (Be specific)

Do you understand the design decisions that were made by your group? (explain if you agree or disagree with the design decisions)

What did you learn during this phase of the project?

Which of the following helped you to understand and complete this phase of the project? (e.g., Textbook, Class videos/PowerPoint, labs, class discussion, group discussions, external sources). If external sources, please specify.

Given the following categories, evaluate yourself and your team member. Estimate the contributions based both in terms of hours and percentages of total work. *(For example: I attended all the meeting 100%, 4 hours, I implemented a small class, but the rest of the classes were implemented by my team members, so I estimate my contribution as 25% and 3 hours, I did most of the documentation, so my contribution was 90% and spent 3 hour)*

YOU	Estimated # of Hours Spent (during the phase)	Estimated % of your contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		

Your Team Member's Name:	Estimated # of Hours Spent (during the phase)	Estimated % of her/his contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		

Your Team Member's Name:	Estimated # of Hours Spent (during the phase)	Estimated % of her/his contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		